

Linux 进程控制函数介绍

进程创建相关的系统调用

1.fork ()

创建一个新的子进程。其子进程会复制父进程的数据与堆栈空间，并继承父进程的用户代码、组代码、环境变量、已打开的文件代码、工作目录和资源限制。

系统调用格式： pid=fork()

参数定义： int fork()

fork()返回值意义如下：

- 0：在子进程中，pid 变量保存的 fork()返回值为 0，表示当前进程是子进程。
- >0：在父进程中，pid 变量保存的 fork()返回值为子进程的 id 值（进程唯一标识符）。
- -1：创建失败。

如果 fork()调用成功，它向父进程返回子进程的 PID，并向子进程返回 0，即 fork()被调用了一次，但返回了两次。此时 OS 在内存中建立一个新进程，所建的新进程是调用 fork()父进程（parent process）的副本，称为子进程（child process）。子进程继承了父进程的许多特性，并具有与父进程完全相同的用户级上下文。父进程与子进程并发执行。

核心为 fork()完成以下操作：

（1）为新进程分配一个进程表项和进程标识符

进入 fork()后，核心检查系统是否有足够的资源来建立一个新进程。若资源不足，则 fork()系统调用失败；否则，核心为新进程分配一进程表项和唯一的进程标识符。

（2）检查同时运行的进程数目

超过预先规定的最大数目时，fork()系统调用失败。

(3) 拷贝进程表项中的数据

将父进程的当前目录和所有已打开的数据拷贝到子进程表项中,并置进程的状态为“创建”状态。

(4) 子进程继承父进程的所有文件

对父进程当前目录和所有已打开的文件表项中的引用计数加 1。

(5) 为子进程创建进程上、下文

进程创建结束,设子进程状态为“内存中就绪”并返回子进程的标识符。

(6) 子进程执行

虽然父进程与子进程程序完全相同,但每个进程都有自己的程序计数器 PC(注意子进程的 PC 开始位置),然后根据 pid 变量保存的 fork()返回值的不同,执行了不同的分支语句。

2. getpid()

取得目前进程的识别码(进程 ID),许多程序利用取到的此值来建立临时文件,以避免临时文件相同带来的问题。

系统调用格式: `int getpid()`

例如: `#include<unistd.h>`

```
main()
```

```
{printf("pid=%d\n",getpid());}
```

3. getppid

取得目前进程的父进程识别码。

系统调用格式: `int getppid()`

例如: `#include<unistd.h>`

```
main()
```

```
{printf("My parent's pid=%d\n",getppid());}
```

在 UNIX/LINUX 中 fork()是一个非常有用的系统调用,但在 UNIX/LINUX

中建立进程除了 `fork()` 之外，也可用与 `fork()` 配合使用的 `exec()`。

4. `exec()` 系列

系统调用 `exec()` 系列，也可用于新程序的运行。`fork()` 只是将父进程的用户级上下文拷贝到新进程中，而 `exec()` 系列可以将一个可执行的二进制文件覆盖在新进程的用户级上下文的存储空间上，以更改新进程的用户级上下文。`exec()` 系列中的系统调用都完成相同的功能，它们把一个新程序装入内存，来改变调用进程的执行代码，从而形成新进程。如果 `exec()` 调用成功，调用进程将被覆盖，然后从新程序的入口开始执行，这样就产生了一个新进程，新进程的进程标识符 `id` 与调用进程相同。

`exec()` 没有建立一个与调用进程并发的子进程，而是用新进程取代了原来进程。所以 `exec()` 调用成功后，没有任何数据返回，这与 `fork()` 不同。`exec()` 系列系统调用在 UNIX 系统库 `unistd.h` 中，共有 `execl`、`execlp`、`execle`、`execv`、`execvp` 五个，其基本功能相同，只是以不同的方式来给出参数。

一种是直接给出参数的指针，如：

```
int  execl(path,arg0,arg1,...argn,0);
char *path,*arg0,*arg1,...,*argn;
```

另一种是给出指向参数表的指针，如：

```
int execv(path,argv);
char *path,*argv[ ];
```

具体使用可参考有关书。

5. `exec()` 和 `fork()` 联合使用

系统调用 `exec` 和 `fork()` 联合使用能为程序开发提供有力支持。用 `fork()` 建立子进程，然后在子进程中使用 `exec()`，这样就实现了父进程与一个与它完全不同子进程的并发执行。

一般，`wait`、`exec` 联合使用的模型为：

```
int status;
.....
if (fork() != 0)
{
    .....;
    execl(...);
```

```
.....;  
}
```

```
wait(&status);
```

6. wait ()

等待子进程运行结束。如果子进程没有完成，父进程一直等待。wait()将调用进程挂起，直至其子进程因暂停或终止而发来软中断信号为止。如果在 wait() 前已有子进程暂停或终止，则调用进程做适当处理后便返回。

系统调用格式：

```
int wait(status)  
int *status;
```

其中，status 是用户空间的地址。它的低 8 位反应子进程状态，为 0 表示子进程正常结束，非 0 则表示出现了各种各样的问题；高 8 位则带回了 exit() 的返回值。exit() 返回值由系统给出。

核心对 wait() 作以下处理：

- (1) 首先查找调用进程是否有子进程，若无，则返回出错码；
- (2) 若找到一处于“僵死状态”的子进程，则将子进程的执行时间加到父进程的执行时间上，并释放子进程的进程表项；
- (3) 若未找到处于“僵死状态”的子进程，则调用进程便在可被中断的优先级上睡眠，等待其子进程发来软中断信号时被唤醒。

7. exit ()

终止进程的执行。

系统调用格式：

```
void exit(status)  
int status;
```

其中，status 是返回给父进程的一个整数，以备查考。

为了及时回收进程所占用的资源并减少父进程的干预，UNIX/LINUX 利用 exit() 来实现进程的自我终止，通常父进程在创建子进程时，应在进程的末尾安排一条 exit()，使子进程自我终止。exit(0) 表示进程正常终止，exit(1) 表示进程运行有错，异常终止。

如果调用进程在执行 exit() 时，其父进程正在等待它的终止，则父进程可立即得到其返回的整数。核心须为 exit() 完成以下操作：

- (1) 关闭软中断
- (2) 回收资源
- (3) 写记帐信息
- (4) 置进程为“僵死状态”